

コンパイラ (3)

2026/10/9

上昇型構文解析

$A \rightarrow \alpha$ の右辺へのマッチングによって A に還元する

出力：最右導出の逆順

入力 (左から) に従って下側から還元

$$\begin{aligned} w &= w_1 w_2 \Leftarrow N_1 w_2 & N_1 \rightarrow w_1 \text{ によって還元} \\ N_1 w_2 &= N_1 w_{21} w_{22} \Leftarrow N_1 N_2 w_{22} \end{aligned}$$

右側の還元が最後になる＝最右導出の逆順

難しい点： $\alpha(i)$ に Σ と N が混在

還元するときのルールの選択がキーポイント

プッシュダウン変換機械 (Pushdown Transducer)

$$T = (\Gamma, \Sigma, O, \Delta, l_0, F)$$

Γ スタック記号の有限集合

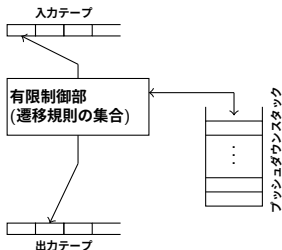
Σ 入力アルファベット

O 出力記号

Δ 遷移関数: $\Gamma^+ \times \Sigma \rightarrow \Gamma^* \times O^* \times \{0, 1\}$

l_0 初期スタック記号

$F \subseteq \Gamma \cup \{\epsilon\}$ 最終スタック記号集合



PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \xrightarrow{a} (w, \pi)$$

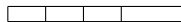
$$\Delta(u, a) = (w, \pi, 0) : u \xrightarrow{(a)} (w, \pi) \quad (\text{先読み動作})$$

$a_1 \ a_2 \ a_3 \ \dots$

有限制御部
(遷移規則の集合)



$(l_0, a_1 a_2 a_3, \varepsilon)$



初期状態をスタックに
入れてスタート

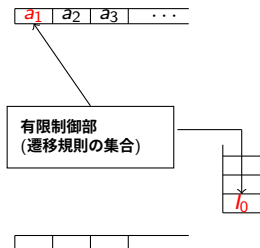
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$l_0 \rightarrow^{a_1} (c_1 c_2, o_1)$$

$$(l_0, a_1 a_2 a_3, \varepsilon)$$

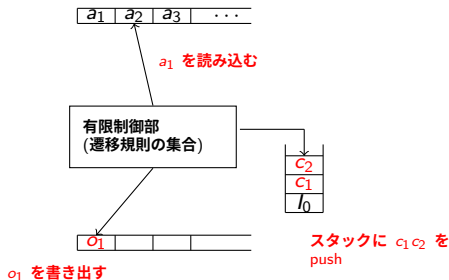
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$l_0 \rightarrow^{a_1} (c_1 c_2, o_1)$$

$$\vdash \begin{array}{l} (l_0, a_1 a_2 a_3, \varepsilon) \\ (l_0 c_1 c_2, a_2 a_3, o_1) \end{array}$$

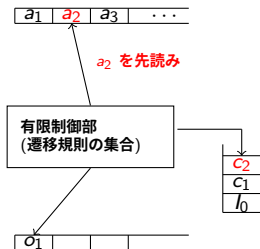
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$\begin{aligned} l_0 &\rightarrow^{a_1} (c_1 c_2, o_1) \\ c_2 &\rightarrow^{(a_2)} (c_3, o_2) \end{aligned}$$

$$\begin{aligned} &(l_0, a_1 a_2 a_3, \varepsilon) \\ \vdash &(l_0 c_1 c_2, a_2 a_3, o_1) \end{aligned}$$

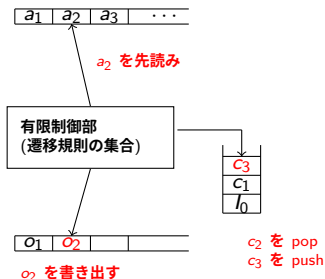
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$l_0 \rightarrow^{a_1} (c_1 c_2, o_1)$$

$$c_2 \rightarrow^{(a_2)} (c_3, o_2)$$

$$(l_0, a_1 a_2 a_3, \varepsilon)$$

$$\vdash (l_0 c_1 c_2, a_2 a_3, o_1)$$

$$\vdash (l_0 c_1 c_3, a_2 a_3, o_1 o_2)$$

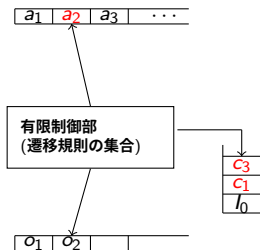
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$\begin{aligned} l_0 &\rightarrow^{a_1} (c_1 c_2, o_1) \\ c_2 &\rightarrow^{(a_2)} (c_3, o_2) \\ c_1 c_3 &\rightarrow^{a_2} (\varepsilon, o_3) \end{aligned}$$

$$\begin{aligned} &(l_0, a_1 a_2 a_3, \varepsilon) \\ \vdash &(l_0 c_1 c_2, a_2 a_3, o_1) \\ \vdash &(l_0 c_1 c_3, a_2 a_3, o_1 o_2) \end{aligned}$$

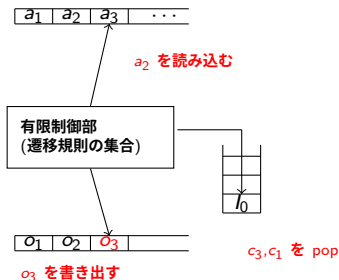
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \xrightarrow{a} (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \xrightarrow{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$l_0 \rightarrow^{a_1} (c_1 c_2, o_1)$$

$$c_2 \rightarrow^{(a_2)} (c_3, o_2)$$

$$c_1 c_3 \rightarrow^{a_2} (\varepsilon, o_3)$$

$$(l_0, a_1 a_2 a_3, \varepsilon)$$

$$\vdash (l_0 c_1 c_2, a_2 a_3, o_1)$$

$$\vdash (l_0 c_1 c_3, a_2 a_3, o_1 o_2)$$

$$\vdash (l_0, a_3, o_1 o_2 o_3)$$

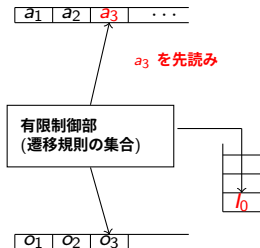
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$\begin{aligned} l_0 &\rightarrow^{a_1} (c_1 c_2, o_1) \\ c_2 &\rightarrow^{(a_2)} (c_3, o_2) \\ c_1 c_3 &\rightarrow^{a_2} (\varepsilon, o_3) \\ l_0 &\rightarrow^{(a_3)} (l_0, \varepsilon) \end{aligned}$$

$$\begin{aligned} &(l_0, a_1 a_2 a_3, \varepsilon) \\ \vdash &(l_0 c_1 c_2, a_2 a_3, o_1) \\ \vdash &(l_0 c_1 c_3, a_2 a_3, o_1 o_2) \\ \vdash &(l_0, a_3, o_1 o_2 o_3) \end{aligned}$$

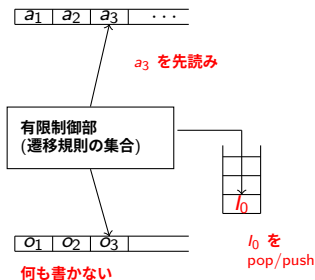
計算履歴

PDT の動作

(スタック, 入力テープ, 出力テープ) : 時点表示 (configuration)

$$\Delta(u, a) = (w, \pi, 1) : u \rightarrow^a (w, \pi)$$

$$\Delta(u, a) = (w, \pi, 0) : u \rightarrow^{(a)} (w, \pi) \quad (\text{先読み動作})$$



$$\begin{aligned} l_0 &\rightarrow^{a_1} (c_1 c_2, o_1) \\ c_2 &\rightarrow^{(a_2)} (c_3, o_2) \\ c_1 c_3 &\rightarrow^{a_2} (\varepsilon, o_3) \\ l_0 &\rightarrow^{(a_3)} (l_0, \varepsilon) \\ &(l_0, a_1 a_2 a_3, \varepsilon) \\ \vdash &(l_0 c_1 c_2, a_2 a_3, o_1) \\ \vdash &(l_0 c_1 c_3, a_2 a_3, o_1 o_2) \\ \vdash &(l_0, a_3, o_1 o_2 o_3) \\ \vdash &(l_0, a_3, o_1 o_2 o_3) \end{aligned}$$

計算履歴

構文木の構成と導出

スタック $l_0(i$

入力 $+n) + i * i\#$

$(i + n) + i * i\#$

$(i$

$E \rightarrow E \underset{1}{+} E \mid E \underset{2}{-} E \mid E \underset{3}{*} E \mid E \underset{4}{/} E \mid (E) \underset{5} \mid i \underset{6} \mid n \underset{7}$

構文木の構成と導出

スタック $I_0(E$

入力 $+n) + i * i \#$

$$(i + n) + i * i \# \Leftarrow_6 (E + n) + i * i \#$$

E
|
(i

$E \rightarrow E + E \mid E - E \mid E * E \mid E / E \mid (E) \mid i \mid n$
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0(E)$

入力 $+i * i \#$

$$\begin{aligned} (i + n) + i * i \# &\Leftarrow_6 (E+n) + i * i \# \\ \Leftarrow_7 (E+E) + i * i \# &\Leftarrow_1 (E) + i * i \# \end{aligned}$$



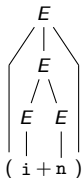
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック I_0E

入力 $+i * i \#$

$(i + n) + i * i \# \Leftarrow_6 (E + n) + i * i \#$
 $\Leftarrow_7 (E + E) + i * i \# \Leftarrow_1 (E) + i * i \#$
 $\Leftarrow_5 E + i * i \#$



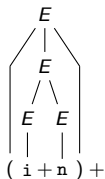
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0 E +$

入力 $i * i \#$

$(i + n) + i * i \# \Leftarrow_6 (E + n) + i * i \#$
 $\Leftarrow_7 (E + E) + i * i \# \Leftarrow_1 (E) + i * i \#$
 $\Leftarrow_5 E + i * i \#$



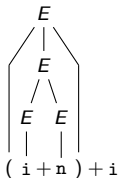
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0E + i$

入力 $*i\#$

$(i + n) + i * i\# \Leftarrow_6 (E+n) + i * i\#$
 $\Leftarrow_7 (E+E) + i * i\# \Leftarrow_1 (E) + i * i\#$
 $\Leftarrow_5 E+i * i\#$



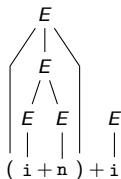
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0E + E$

入力 $*i\#$

$(i + n) + i * i\# \Leftarrow_6 (E+n) + i * i\#$
 $\Leftarrow_7 (E+E) + i * i\# \Leftarrow_1 (E) + i * i\#$
 $\Leftarrow_5 E+i * i\# \quad \Leftarrow_6 E+E*i\#$



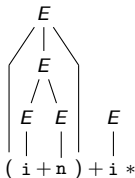
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0E + E^*$

入力 $i\#$

$(i + n) + i * i\# \xleftarrow{6} (E+n) + i * i\#$
 $\xleftarrow{7} (E+E) + i * i\# \xleftarrow{1} (E) + i * i\#$
 $\xleftarrow{5} E+i * i\# \xleftarrow{6} E+E*i\#$



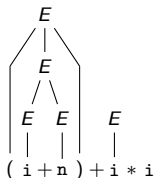
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0E + E * i$

入力 $\#$

$(i + n) + i * i \# \xleftarrow{6} (E + n) + i * i \#$
 $\xleftarrow{7} (E + E) + i * i \# \xleftarrow{1} (E) + i * i \#$
 $\xleftarrow{5} E + i * i \# \xleftarrow{6} E + E * i \#$



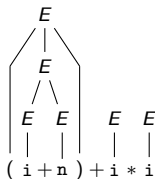
$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック $I_0 E + E * E$

入力 $\#$

$(i + n) + i * i \# \Leftarrow_6 (E + n) + i * i \#$
 $\Leftarrow_7 (E + E) + i * i \# \Leftarrow_1 (E) + i * i \#$
 $\Leftarrow_5 E + i * i \# \quad \Leftarrow_6 E + E * i \#$
 $\Leftarrow_6 E + E * E \#$



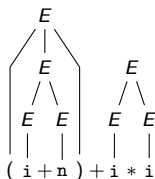
$E \rightarrow E + E$ | $E \rightarrow E - E$ | $E \rightarrow E * E$ | $E \rightarrow E / E$ | $E \rightarrow (E)$ | $E \rightarrow i$ | $E \rightarrow n$
1 2 3 4 5 6 7

構文木の構成と導出

スタック $_0E + E$

入力 $\#$

$(i + n) + i * i \# \Leftarrow_6 (E + n) + i * i \#$
 $\Leftarrow_7 (E + E) + i * i \# \Leftarrow_1 (E) + i * i \#$
 $\Leftarrow_5 E + i * i \# \Leftarrow_6 E + E * i \#$
 $\Leftarrow_6 E + E * E \# \Leftarrow_3 E + E \#$

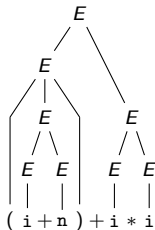


$E \rightarrow E + E$ | $E - E$ | $E * E$ | E / E | (E) | i | n
1 2 3 4 5 6 7

構文木の構成と導出

スタック I_0E

入力 #



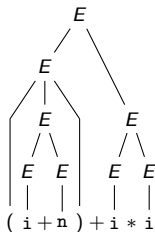
$(i + n) + i * i \# \Leftarrow_6 (E+n) + i * i \#$
 $\Leftarrow_7 (E+E) + i * i \# \Leftarrow_1 (E) + i * i \#$
 $\Leftarrow_5 E+i * i \# \qquad \Leftarrow_6 E+E * i \#$
 $\Leftarrow_6 E+E * E \# \qquad \Leftarrow_3 E+E \#$
 $\Leftarrow_1 E \#$

$E \rightarrow E + E \quad | \quad E - E \quad | \quad E * E \quad | \quad E / E \quad | \quad (E) \quad | \quad i \quad | \quad n$
1
2
3
4
5
6
7

構文木の構成と導出

スタック I_0

入力



$(i + n) + i * i \# \Leftarrow_6 (E + n) + i * i \#$
 $\Leftarrow_7 (E + E) + i * i \# \Leftarrow_1 (E) + i * i \#$
 $\Leftarrow_5 E + i * i \# \qquad \Leftarrow_6 E + E * i \#$
 $\Leftarrow_6 E + E * E \# \qquad \Leftarrow_3 E + E \#$
 $\Leftarrow_1 E \#$

$E \rightarrow E + E \quad | \quad E - E \quad | \quad E * E \quad | \quad E / E \quad | \quad (E) \quad | \quad i \quad | \quad n$
1 2 3 4 5 6 7

PDT としての動作

$(l_0, (i + n) + i * i\#, \varepsilon)$	
⊢ $(l_0, i + n) + i * i\#, \varepsilon)$	(シフトルール)
⊢ $(l_0(i, +n) + i * i\#, \varepsilon)$	$\varepsilon \rightarrow ($ $('', \varepsilon)$
⊢ $(l_0(E, +n) + i * i\#, 6)$	$\varepsilon \rightarrow)$ $(')', \varepsilon)$
⊢ $(l_0(E+, n) + i * i\#, 6)$	$\varepsilon \rightarrow i$ $('i', \varepsilon)$
⊢ $(l_0(E + n,) + i * i\#, 6)$	$\varepsilon \rightarrow^n$ $('n', \varepsilon)$
⊢ $(l_0(E + E,) + i * i\#, 67)$	$\varepsilon \rightarrow +$ $('+', \varepsilon)$
⊢ $(l_0(E,) + i * i\#, 671)$	$\varepsilon \rightarrow^*$ $('*', \varepsilon)$
⊢ $(l_0(E), +i * i\#, 671)$	
⊢ $(l_0E, +i * i\#, 6715)$	(還元ルール)
⊢ $(l_0E+, i * i\#, 6715)$	$E' + E \rightarrow (E, 1)$
⊢ $(l_0E + i, *i\#, 6715)$	$E' - E \rightarrow (E, 2)$
⊢ $(l_0E + E, *i\#, 67156)$	$E' * E \rightarrow (E, 3)$
⊢ $(l_0E + E*, i\#, 67156)$	$E' / E \rightarrow (E, 4)$
⊢ $(l_0E + E * i, \#, 67156)$	$'(E)'$ $\rightarrow (E, 5)$
⊢ $(l_0E + E * E, \#, 671566)$	$i \rightarrow (E, 6)$
⊢ $(l_0E + E, \#, 6715663)$	$n \rightarrow (E, 7)$
⊢ $(l_0E, \#, 67156631)$	

このままでは、いつどのルールを適用するのがいいのかわからない

上向き構文解析の方針

- ▶ スタックに入力記号を貯める (シフトルール)
- ▶ 生成規則の右辺にマッチしたら、左辺の非終端記号に置き換える。(還元ルール)

上向き構文解析の問題点：還元ルールが適用できるときに還元ルールを適用すべき？

$(i+n)+i*i$ の解析において：

$(i+n)+i$ まで解析したとき、規則 1 を適用するか、 $*$ をスタックに push するか？

$w \in L(G)$ という問題では同じだが、構成される構文木が異なる

拡大文法

$S' \notin N$ である非終端記号 S' と生成規則 $\{S' \rightarrow S\}$ を追加
PDT の最終時点表示を明確にする。

先の例

$\vdash (l_0 E, \#, 67156631)$

$\vdash (l_0 S', \#, 67156631)$

遷移規則 $E \xrightarrow{(\#)} (S', 0)$ を適用

時点表示が $(l_0 S', \#, \dots)$ の形になったら停止

LR 構文解析

PDT を用いた上昇型構文解析：最右導出の逆順を求める

先の例：(無あいまいであれば)13665176 の順で生成規則を適用する最右導出によって解析木を構成可能

$$A \rightarrow \frac{\alpha_1 \cdots \alpha_n}{\text{左からマッチングをとる}}$$

$\alpha_i \in \Sigma$ のとき、入力文字を読みこむ

Shift 動作

$\alpha_i \in N$ のとき、一旦 α_i をスタックにつんで再帰的にマッチング動作を繰り返す

最終的に α_i がマッチすることがわかれば還元

Reduce 動作

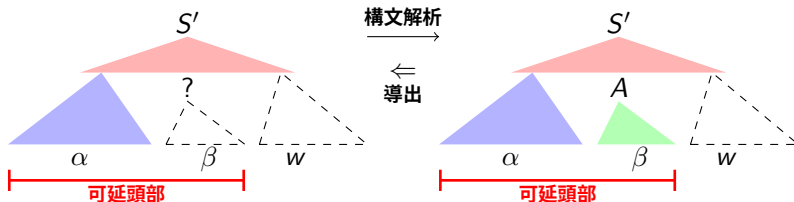
この規則の最後までいけば全体を A に還元する

可延頭部 (viable prefix)

CF 文法 G による最右導出： $S' \Rightarrow_{rm}^* \alpha A w \Rightarrow_{rm} \alpha \beta w$ ($w \in \Sigma^* \#$)

G の可延頭部 (viable prefix) = αA , $\alpha \beta$ の接頭語

一般的な上向き構文解析：可延頭部の木構造を構成していく



右辺が β にマッチする規則を選んで左辺 A に置換

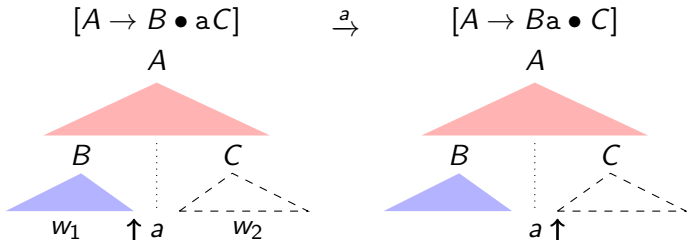
LR(0) 項

各生成規則のどこまでを解析したかマークをつける

$A \rightarrow \alpha_1 \alpha_2 \cdots \alpha_j \cdots \alpha_n$ から以下の LR(0) 項を生成する。

$$\begin{aligned} & [A \rightarrow \bullet \alpha_1 \alpha_2 \cdots \alpha_j \cdots \alpha_n] \\ & [A \rightarrow \alpha_1 \bullet \alpha_2 \cdots \alpha_j \cdots \alpha_n] \\ & \vdots \\ & [A \rightarrow \alpha_1 \alpha_2 \cdots \bullet \alpha_j \cdots \alpha_n] \\ & [A \rightarrow \alpha_1 \alpha_2 \cdots \alpha_j \bullet \cdots \alpha_n] \\ & \vdots \\ & [A \rightarrow \alpha_1 \alpha_2 \cdots \alpha_j \cdots \bullet \alpha_n] \\ & [A \rightarrow \alpha_1 \alpha_2 \cdots \alpha_j \cdots \alpha_n \bullet] \end{aligned}$$

LR(0) 項と構文解析



LR(0) 項間の遷移

LR(0) 項の意味：

その生成規則が適用されて、**•** までは構文解析済みである状態を表す。

構文解析 = LR(0) 項における **• を次に進める**

定義 4.12

LR(0) 項間の遷移関係

$$[A \rightarrow \alpha_1 \alpha_2 \cdots \bullet X \cdots \alpha_n] \xrightarrow{X} [A \rightarrow \alpha_1 \alpha_2 \cdots X \bullet \cdots \alpha_n]$$

$$X \rightarrow \gamma \in P \text{ のとき } [A \rightarrow \beta_1 \bullet X \beta_2] \xrightarrow{\varepsilon} [X \rightarrow \bullet \gamma]$$

LR(0) 項に基づく構文解析の例

$$G_{Arith} = \langle \{S', E, T, F\}, \{i, +, *, (,)\}, P, S' \rangle$$

$$P = \left\{ \begin{array}{lcl} 0: & S' & \rightarrow E\# \\ 1: & E & \rightarrow E + T \\ 2: & E & \rightarrow T \\ 3: & T & \rightarrow T * F \\ 4: & T & \rightarrow F \\ 5: & F & \rightarrow (E) \\ 6: & F & \rightarrow i \end{array} \right\}$$

LR(0) 項に基づく構文解析の例

$$G_{Arith} = \langle \{S', E, T, F\}, \{i, +, *, (,)\}, P, S' \rangle$$

LR(0) 項の生成：

$[S' \rightarrow \bullet E \#], [S' \rightarrow E \bullet \#]$

$[E \rightarrow \bullet E + T], [E \rightarrow E \bullet + T], [E \rightarrow E + \bullet T], [E \rightarrow E + T \bullet]$

$[E \rightarrow \bullet T], [E \rightarrow T \bullet]$

$[T \rightarrow \bullet T * F], [T \rightarrow T \bullet * F], [T \rightarrow T * \bullet F], [T \rightarrow T * F \bullet]$

$[T \rightarrow \bullet F], [T \rightarrow F \bullet]$

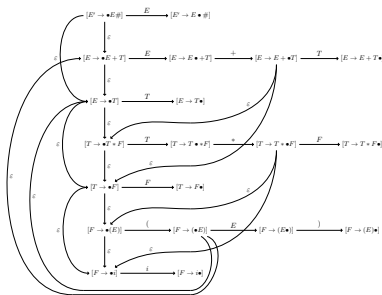
$[F \rightarrow \bullet (E)], [F \rightarrow (\bullet E)], [F \rightarrow (E \bullet)], [F \rightarrow (E) \bullet]$

$[F \rightarrow \bullet i], [F \rightarrow i \bullet]$

LR(0) 項に基づく構文解析の例

$$G_{Arith} = \langle \{S', E, T, F\}, \{i, +, *, (,)\}, P, S' \rangle$$

遷移規則から LR(0) オートマトンを構成：



NFA なので DFA に変換=可延頭部 α でまとめる

LR(0) 項に基づく構文解析の例

$$G_{Arith} = \langle \{S', E, T, F\}, \{i, +, *, (,)\}, P, S' \rangle$$

正準 LR(0) 項：初期状態から $\alpha = X_1 \cdots X_n$ でたどれる LR 項に一致

($X_i \in \Sigma \cup N$)

$$\left\{ \begin{array}{l} l_0 = \left\{ [S' \rightarrow \bullet E \#], [E \rightarrow \bullet E + T], [E \rightarrow \bullet T], [T \rightarrow \bullet T * F], [T \rightarrow \bullet F], \right. \\ \left. [F \rightarrow \bullet (E)], [F \rightarrow \bullet i] \right\}, \\ l_1 = \left\{ [S' \rightarrow E \bullet \#], [E \rightarrow E \bullet + T] \right\}, \quad l_2 = \left\{ [E \rightarrow T \bullet], [T \rightarrow T \bullet * F] \right\}, \\ l_3 = \left\{ [T \rightarrow F \bullet] \right\}, \\ l_4 = \left\{ [F \rightarrow (\bullet E)], [E \rightarrow \bullet E + T], [E \rightarrow \bullet T], [T \rightarrow \bullet T * F], [T \rightarrow \bullet F], \right. \\ \left. [F \rightarrow \bullet (E)], [F \rightarrow \bullet i] \right\}, \\ l_5 = \left\{ [F \rightarrow i \bullet] \right\}, \\ l_6 = \left\{ [E \rightarrow E + \bullet T], [T \rightarrow \bullet T * F], [T \rightarrow \bullet F], [F \rightarrow \bullet (E)], [F \rightarrow \bullet i] \right\}, \\ l_7 = \left\{ [T \rightarrow T * \bullet F], [F \rightarrow \bullet (E)], [F \rightarrow \bullet i] \right\}, \\ l_8 = \left\{ [F \rightarrow (E \bullet)], [E \rightarrow E \bullet + T] \right\}, \\ l_9 = \left\{ [E \rightarrow E + T \bullet], [T \rightarrow T \bullet * F] \right\}, \\ l_{10} = \left\{ [T \rightarrow T * F \bullet] \right\}, \quad l_{11} = \left\{ [F \rightarrow (E) \bullet] \right\} \end{array} \right\}$$

正準 LR(0) 遷移系の構成

以下の手順に沿って構成される遷移系 $\Theta = (S, \varepsilon)$ を正準 LR(0) 遷移系とよぶ. S は Θ の状態, ε は Θ の辺である.

ステップ 1: $I_0 = \text{Goto}(\{[S' \rightarrow \bullet S\#]\}, \varepsilon)$ とする.

ステップ 2: $\Theta \leftarrow (\{I_0\}, \emptyset); T \leftarrow \{I_0\}$

ステップ 3: $T = \emptyset$ ならば終了する. Θ に正準 LR(0) 遷移系が得られる.

ステップ 4: $I \in T$ を選び, T から削除する.

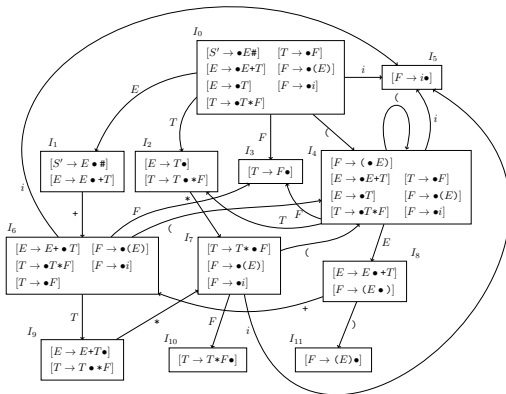
ステップ 5: すべての $M \in \text{act}(I)$ に対して,
 $I' = \{\theta' \mid \theta \in I, \theta \xrightarrow{\varepsilon^*} \xrightarrow{M} \xrightarrow{\varepsilon^*} \theta'\}$ を構成する.

$I' \notin S$ ならば, T に I' を, S に I' を追加し, ε に $I \xrightarrow{M} I'$ を追加する.

$I' \in S$ ならば, ε に $I \xrightarrow{M} I'$ を追加する.

ステップ 6: ステップ 3 へもどる.

G_{Arith} の正準 LR(0) 遷移系



正準 LR(0) 遷移系 $\rightarrow$ PDT

$\mathcal{I}$: LR(0) 項全体の集合, $\mathcal{P}(\mathcal{I})$: LR(0) 項集合の集合

$\text{Goto} : \mathcal{P}(\mathcal{I}) \times (N \cup \Sigma)^* \rightarrow \mathcal{P}(\mathcal{I})$

LR(0) オートマトンをたどってたどりつく正準 LR(0) オートマトンの状態

定義 4.13

I を LR(0) 項の集合とする。

$$\text{Goto}(I, \alpha) = \begin{cases} \{\theta' \mid \theta \xrightarrow{\varepsilon}^* \theta' \text{ となる } \theta \in I \text{ が存在する} \} & \alpha = \varepsilon \text{ のとき} \\ \{\theta' \mid \theta \xrightarrow{M} \theta' \text{ となる } \theta \in \text{Goto}(I, \beta) \text{ が存在する} \} & \alpha = \beta M, M \neq \varepsilon \text{ のとき} \end{cases}$$

$\text{Goto}(I, \alpha) = \text{Goto}(I, \alpha\varepsilon) = \text{Goto}(I, \varepsilon\alpha)$ である

可延頭部 α に対して、 $I(\alpha) = \text{Goto}(\{[S' \rightarrow \bullet S\#]\}, \alpha)$

例えば、 $I(\varepsilon) = I_0$, $I(E+(E)) = I_8$

PDT の動作

初期スタック: $I(\varepsilon) = I_0$

から始めて、シフトルール、還元ルール、受理ルールを適用

底	$I(\varepsilon)$	X_1	$I(X_1)$	X_2	$I(X_1 X_2)$	$\dots$	X_n	$I(X_1 \dots X_n)$	トップ
	状態	記号	状態	記号	状態		記号	状態	

状態と記号を交互にスタックに積む。最後は状態

正準 LR 遷移系 $\rightarrow$ PDT

スタックトップ $l_{top} = l(X_1 \dots X_n) : LR(0)$ オートマトンの状態

還元 $[A \rightarrow \alpha\bullet] \in I_{top}$ のとき：
$$\beta\alpha w \Leftarrow_{rm} \beta A w \text{ (ここで } i : A \rightarrow \alpha \text{)}$$

$$(l_0 \cdots X_{n-k} l_\ell \quad \underbrace{X_{n-k+1} \cdots X_n l_n}_{\alpha}, w\#, o)$$

正準 LR 遷移系 $\rightarrow$ PDT

シフト $[A \rightarrow \beta \bullet a\gamma] \in I_{top}$ のとき

$$(I(\varepsilon) \dots X_n I_n, aX, \pi) \vdash (I(\varepsilon) \dots X_n I_n a I_m, X, \pi)$$

ここで、 $I_m = Goto(I_n, a)$

受理 $[S' \rightarrow S \bullet \#] \in I_{top}$ ならば受理

$$(|S'|, \#, \pi) \vdash (acc, \varepsilon, \pi 0)$$

誤り 上記いずれにもあてはまらないとき